

사내 Java 레거시 코드 분석 도구

[RetroJS]

레거시 코드 이해부터 인수인계 문서화까지

맹재언

Frontend: React + Vite | Backend: Node.js + Express | AST: java-parser

왜 만들었나요?



문서 부족

레거시 코드에 문서가
없거나 부족함



구조 파악 어려움

클래스/메서드 간
관계 파악에 시간 소요



수정 불안감

코드 수정 시 영향
범위를 알 수 없음

결과: 유지보수 시간 증가 · 코드 수정에 대한 불안감 · 인수인계 어려움

Claude/ChatGPT 대신 자체 도구를 쓰는 이유



보안

사내 코드를 외부 서버로
전송하지 않아 코드 유출 위험 없음



정확도

외부 AI는 회사 코드 컨벤션을
모름. AST 정적 분석은 정확함



자동화

반복적인 분석 작업을 자동화하여
개발자 시간 절약



문서화

인수인계 문서를 자동으로
생성하여 즉시 활용 가능

핵심 기능 (MVP)

F-01

코드 입력 및 분석 요청

Java 코드 입력 → 백엔드 API 호출 → 결과 출력

F-02

클래스/메서드 구조 추출

AST 파싱으로 클래스, 메서드 목록, 위치 자동 추출

F-03

외부 메서드 호출 표시

내부 호출과 외부 클래스 호출을 (외부) 표시로 구분

F-04

영향도 분석

메서드 클릭 시 해당 메서드를 호출하는 메서드 하이라이트

F-05

인수인계 문서 자동 생성

분석 결과 기반 문서 자동 생성 + 클립보드 복사

F-06

분석 범위 안내

분석 가능/불가능 항목을 별도 탭으로 안내

실행 화면

[RetroJS] Java 레거시 코드 분석 도구

코드입력

Java 단일 파일

코드를 입력하고 분석 요청을 눌러주세요.

분석 요청

```
1 public class UserService {
2
3     private String userData;
4     private boolean isLoggedIn;
5
6     public boolean validateUser(String name, String email) {
7         if (name == null || email == null) {
8             return false;
9         }
10        return checkEmailFormat(email);
11    }
12
13    private boolean checkEmailFormat(String email) {
14        return email.contains("@") && email.contains(".");
15    }
16
17    public String registerUser(String name, String email) {
18        if (!validateUser(name, email)) {
19            return redirectLogin();
20        }
21        saveToDatabase(name, email);
22        return notifyUser(email);
23    }
24
25    private String redirectLogin() {
26        return "로그인 페이지로 이동합니다.";
27    }
28
29    private void saveToDatabase(String name, String email) {
30        System.out.println("DB 저장: " + name + ", " + email);
31    }
32
33    private String notifyUser(String email) {
34        return "이메일 발송: " + email;
35    }
36 }
```

실행 화면

[RetroJS] Java 레거시 코드 분석 도구

코드입력

Java 단일 파일

```
1 public class UserService {
2
3     private String userData;
4     private boolean isLoggedIn;
5
6     public boolean validateUser(String name, String email) {
7         if (name == null || email == null) {
8             return false;
9         }
10        return checkEmailFormat(email);
11    }
12
13    private boolean checkEmailFormat(String email) {
14        return email.contains("@") && email.contains(".");
15    }
16
17    public String registerUser(String name, String email) {
18        if (!validateUser(name, email)) {
19            return redirectLogin();
20        }
21        saveToDatabase(name, email);
22        return notifyUser(email);
23    }
24
25    private String redirectLogin() {
26        return "로그인 페이지로 이동합니다.";
27    }
28
29    private void saveToDatabase(String name, String email) {
30        System.out.println("DB 저장: " + name + ", " + email);
31    }
32
33    private String notifyUser(String email) {
34        return "이메일 발송: " + email;
35    }
36 }
```

분석 요청

메서드 목록

호출 관계

영향도

인수관계 문서

분석 범위

클래스

UserService

메서드 목록 — 클릭하면 영향도를 확인할 수 있어요

validateUser	6 ~ 11번 줄
checkEmailFormat	13 ~ 15번 줄
registerUser	17 ~ 23번 줄
redirectLogin	25 ~ 27번 줄
saveToDatabase	29 ~ 31번 줄
notifyUser	33 ~ 35번 줄

실행 화면

[RetroJS] Java 레거시 코드 분석 도구

코드입력

Java 단일 파일

```
1 public class UserService {
2
3     private String userData;
4     private boolean isLoggedIn;
5
6     public boolean validateUser(String name, String email) {
7         if (name == null || email == null) {
8             return false;
9         }
10        return checkEmailFormat(email);
11    }
12
13    private boolean checkEmailFormat(String email) {
14        return email.contains("@") && email.contains(".");
15    }
16
17    public String registerUser(String name, String email) {
18        if (!validateUser(name, email)) {
19            return redirectLogin();
20        }
21        saveToDatabase(name, email);
22        return notifyUser(email);
23    }
24
25    private String redirectLogin() {
26        return "로그인 페이지로 이동합니다.";
27    }
28
29    private void saveToDatabase(String name, String email) {
30        System.out.println("DB 저장: " + name + ", " + email);
31    }
32
33    private String notifyUser(String email) {
34        return "이메일 발송: " + email;
35    }
36 }
```

분석 요청

메서드 목록

호출 관계

영향도

인수인계 문서

분석 범위

선택된 메서드 validateUser

영향받는 메서드 registerUser

영향받음

실행 화면

[RetroJS] Java 레거시 코드 분석 도구

코드입력

Java 단일 파일

```
1 public class UserService {
2
3     private String userData;
4     private boolean isLoggedIn;
5
6     public boolean validateUser(String name, String email) {
7         if (name == null || email == null) {
8             return false;
9         }
10        return checkEmailFormat(email);
11    }
12
13    private boolean checkEmailFormat(String email) {
14        return email.contains("@") && email.contains(".");
15    }
16
17    public String registerUser(String name, String email) {
18        if (!validateUser(name, email)) {
19            return redirectLogin();
20        }
21        saveToDatabase(name, email);
22        return notifyUser(email);
23    }
24
25    private String redirectLogin() {
26        return "로그인 페이지로 이동합니다.";
27    }
28
29    private void saveToDatabase(String name, String email) {
30        System.out.println("DB 저장: " + name + ", " + email);
31    }
32
33    private String notifyUser(String email) {
34        return "이메일 발송: " + email;
35    }
36 }
```

분석 요청

메서드 목록

호출 관계

영향도

인수연계 문서

분석 범위

validateUser → checkEmailFormat

checkEmailFormat → contains (외부)

registerUser → validateUser, redirectLogin, saveToDatabase, notifyUser

redirectLogin 호출 없음

saveToDatabase → println (외부)

notifyUser 호출 없음

실행 화면

[RetroJS] Java 레거시 코드 분석 도구

코드입력

Java 단일 파일

메서드 목록호출 관계영향도인수인계 문서분석 범위

클립보드 복사

```
1 public class UserService {
2
3     private String userData;
4     private boolean isLoggedIn;
5
6     public boolean validateUser(String name, String email) {
7         if (name == null || email == null) {
8             return false;
9         }
10        return checkEmailFormat(email);
11    }
12
13    private boolean checkEmailFormat(String email) {
14        return email.contains("@") && email.contains(".");
15    }
16
17    public String registerUser(String name, String email) {
18        if (!validateUser(name, email)) {
19            return redirectLogin();
20        }
21        saveToDatabase(name, email);
22        return notifyUser(email);
23    }
24
25    private String redirectLogin() {
26        return "로그인 페이지로 이동합니다.";
27    }
28
29    private void saveToDatabase(String name, String email) {
30        System.out.println("DB 저장: " + name + ", " + email);
31    }
32
33    private String notifyUser(String email) {
34        return "이메일 발송: " + email;
35    }
36 }
```

분석 요청

인수인계 문서

클래스 목록

- UserService

주요 메서드 목록

validateUser

- 위치: 6번 줄 ~ 11번 줄

- 호출하는 메서드: checkEmailFormat

checkEmailFormat

- 위치: 13번 줄 ~ 15번 줄

- 호출하는 메서드: contains (외부)

registerUser

- 위치: 17번 줄 ~ 23번 줄

- 호출하는 메서드: validateUser, redirectLogin, saveToDatabase, notifyUser

redirectLogin

- 위치: 25번 줄 ~ 27번 줄

- 호출하는 메서드: 없음

saveToDatabase

- 위치: 29번 줄 ~ 31번 줄

- 호출하는 메서드: println (외부)

notifyUser

- 위치: 33번 줄 ~ 35번 줄

- 호출하는 메서드: 없음

메서드 간 호출 관계

- 'validateUser' → 'checkEmailFormat'

- 'checkEmailFormat' → 'contains' (외부)

- 'registerUser' → 'validateUser', 'redirectLogin', 'saveToDatabase', 'notifyUser'

실행 화면

[RetroJS] Java 레거시 코드 분석 도구

코드입력

Java 단일 파일

```
1 public class UserService {
2
3     private String userData;
4     private boolean isLoggedIn;
5
6     public boolean validateUser(String name, String email) {
7         if (name == null || email == null) {
8             return false;
9         }
10        return checkEmailFormat(email);
11    }
12
13    private boolean checkEmailFormat(String email) {
14        return email.contains("@") && email.contains(".");
15    }
16
17    public String registerUser(String name, String email) {
18        if (!validateUser(name, email)) {
19            return redirectLogin();
20        }
21        saveToDatabase(name, email);
22        return notifyUser(email);
23    }
24
25    private String redirectLogin() {
26        return "로그인 페이지로 이동합니다.";
27    }
28
29    private void saveToDatabase(String name, String email) {
30        System.out.println("DB 저장: " + name + ", " + email);
31    }
32
33    private String notifyUser(String email) {
34        return "이메일 발송: " + email;
35    }
36 }
```

메서드 목록

호출 관계

영향도

인수인계 문서

분석 범위

✓ 분석 가능한 항목

단일 Java 파일 내 클래스 구조
클래스 내 메서드 목록 및 위치
같은 파일 내 메서드 간 직접 호출 관계
객체르 통한 메서드 호출 (같은 파일 내)
메서드 수정/상재 시 영향 받는 메서드
외부 클래스 메서드 호출 표시 (외부 표시)

✗ 분석 불가능한 항목

여러 파일 간 의존성 분석
외부 클래스 메서드의 내부 동작
인터페이스, 추상 클래스 구조
람다식, 스트림 내부 호출
리플렉션을 통한 동적 호출
상속 관계 분석

⚠ 주의 사항

분석 범위는 단일 파일로 제한됩니다.
(외부) 표시는 현재 파일 외부 클래스의 메서드 호출입니다.
AI 설명 기능 연동 후 분석 범위가 확장될 예정입니다.

분석 요청

기술 스택 & 아키텍처



Frontend

- › React + Vite (포트 5173)
- › @monaco-editor/react
- › Java 선택스 하이라이팅
- › 탭 기반 결과 패널 UI
- › 백엔드 API 호출 (fetch)



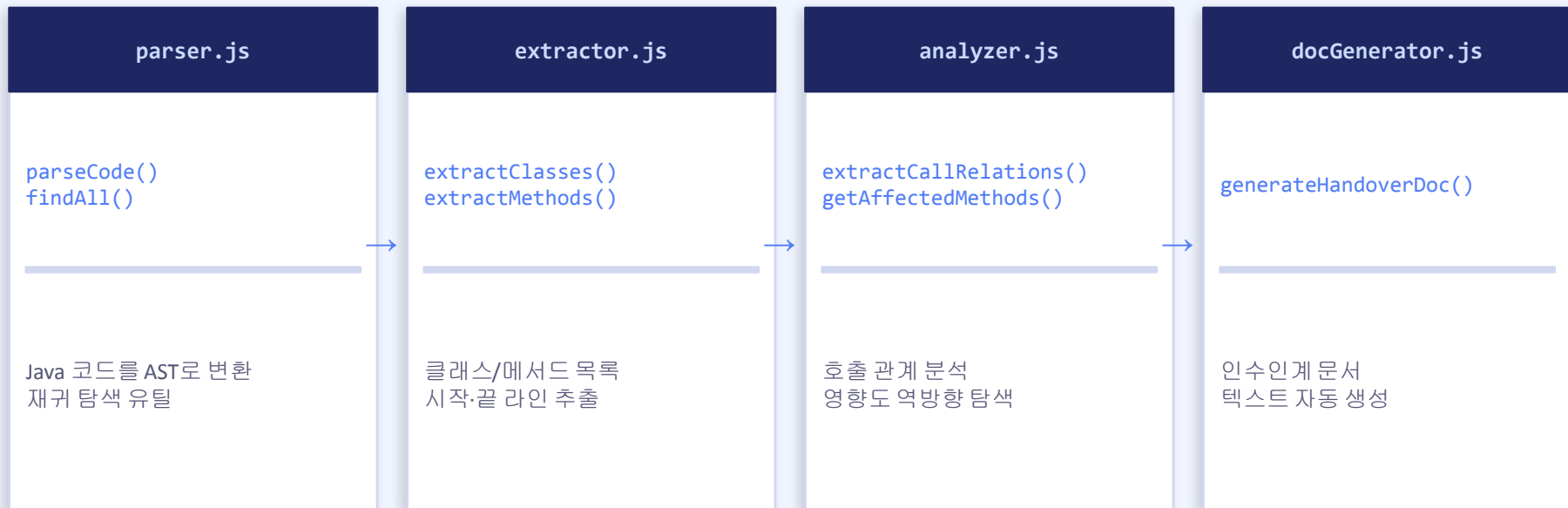
Backend

- › Node.js + Express (포트 3001)
- › java-parser (AST 파싱)
- › parser.js / extractor.js
- › analyzer.js / docGenerator.js
- › dotenv (환경변수 관리)

REST
API

백엔드 모듈 설계

역할 분리 원칙: 프론트엔드 = 시각적 요소만 | 백엔드 = 모든 데이터 처리



코드 분석 흐름



영향도 분석 흐름

메서드 클릭 → POST /api/affected → callRelations 역방향 탐색 → 영향받는 메서드 반환 → 하이라이트 표시

분석 범위

✓ 분석 가능

- › 단일 Java 파일 내 클래스 구조
- › 클래스 내 메서드 목록 및 위치
- › 같은 파일 내 메서드 간 호출 관계
- › 객체를 통한 메서드 호출 (같은 파일)
- › 메서드 수정/삭제 시 영향받는 메서드
- › 외부 클래스 메서드 호출 (외부) 표시

✗ 분석 불가능

- › 여러 파일 간 의존성 분석
- › 외부 클래스 메서드의 내부 동작
- › 인터페이스, 추상 클래스, enum 구조
- › 랴다식, 스트림 내부 호출
- › 리플렉션을 통한 동적 호출
- › 상속 관계 분석

향후 개발 계획 (AI API 확보 후)



F-07 — 코드 자연어 설명

Claude API 연동하여 클래스/메서드 역할을 한국어로 자동 설명



F-08 — 개선 및 리팩토링 제안

순환 복잡도(Cyclomatic Complexity) 수치 기반 리팩토링 방향 제시

감사합니다

단순 코드 설명을 넘어,
AST 기반 정적 분석으로 Java 레거시 코드의 이해부터
인수인계 문서화까지 지원하는 사내 전용 코드 분석 도구

[RetroJS]